

SAFELINE WAF SECURITY LAB

Web Application Firewall Deployment and Security Testing

PREPARED BY

AASHISH KARKI

Cybersecurity Enthusiast

LAB ENVIRONMENT

Kali Linux VM - Security Testing & Attack Simulation

Ubuntu Server VM - SafeLine WAF, Apache & DVWA

SafeLine WAF - Web Application Firewall & Reverse Proxy

DVWA - Deliberately Vulnerable Web Application

HTTPS / TLS - Secure Client-to-WAF Communication

VMware - Virtualized Security Laboratory

Table of Contents

Abstract.....	1
1. Introduction.....	1
2. Objectives	1
4. Scope.....	2
5. Lab Environment	2
6. Software and Technologies.....	2
7. Methodology.....	3
8. Implementation and Configuration.....	3
8.1 Apache and DVWA Deployment.....	3
8.2 DVWA Application Configuration	4
8.3 Moving Apache to Port 8080	5
8.4 Local DNS/Hostname Resolution	6
8.5 Self-Signed HTTPS Certificate.....	7
8.6 SafeLine WAF Installation	8
8.7 SafeLine Reverse Proxy Configuration	9
8.8 Final HTTPS Access	10
9. Security Testing	11
9.1 Baseline Application Test	11
9.2 SQL Injection Test	11
9.3 HTTP Flood Defense	13
9.4 Custom IP Deny Rule.....	13
10. Results and Analysis.....	15
11. Security Architecture and Defense Flow	15
12. Limitations.....	15
13. Conclusion	16
14. Future Work.....	16
15. References.....	16

List of figures

Figure 1: Lab architecture.....	2
Figure 2: Apache on Ubuntu server.....	4
Figure 3: DVWA access before WAF	5
Figure 4: Apache on 8080 port	5
Figure 5: DVWA on 8080 port.....	6
Figure 6: DVWA dns resolution from kali	7
Figure 7: DVWA ssl certificate	7
Figure 8: Safeline installation.....	8
Figure 9: Safeline Docker container	8
Figure 10: Safeline DVWA application config	9
Figure 11: SSL certificates imported on DVWA	10
Figure 12: DVWA safeline protected	10
Figure 13: DVWA through safeline	11
Figure 14: SQL injection attempt	12
Figure 15: Safeline SQL alert	12
Figure 16: Safeline flood detection.....	13
Figure 17: Custom IP deny rule.....	14
Figure 18 : Reuest blocked by IP deny rule.....	14

Abstract

This project presents a controlled cybersecurity laboratory built to demonstrate how a Web Application Firewall (WAF) can protect a deliberately vulnerable web application. The laboratory uses Damn Vulnerable Web Application (DVWA) hosted on an Ubuntu Server, Apache as the web server, MySQL as the database, Kali Linux as the testing platform, and SafeLine WAF as the reverse-proxy security layer. Apache was separated from the WAF by moving the DVWA backend to TCP port 8080, while SafeLine exposed the application through HTTPS on port 443. A local hostname, dvwa.local, and a self-signed TLS certificate were used for the lab. Security testing demonstrated SQL injection detection and blocking, along with advanced SafeLine controls including HTTP flood protection and custom IP denial. The project shows the relationship between a vulnerable application, an attack source, a reverse proxy, and a defensive WAF in an isolated laboratory environment.

1. Introduction

Web applications are frequently exposed to attacks such as SQL injection, excessive request traffic, and unauthorized sources. A WAF provides an application-layer security control that can inspect HTTP requests before they reach a web application. This project builds a small laboratory environment in which DVWA intentionally provides vulnerable application behavior and SafeLine is positioned in front of it to demonstrate detection and blocking.

The project is based on the supplied SafeLine WAF laboratory guide, whose stated goals include deploying DVWA on Ubuntu, demonstrating SQL injection from Kali Linux, showing SafeLine protection, and exploring HTTP flood defense and custom deny rules.

2. Objectives

- Deploy DVWA on an Ubuntu Server using Apache, PHP, and MySQL.
- Move the Apache/DVWA backend to port 8080 so SafeLine can operate as the front-end security layer.
- Configure the local hostname dvwa.local for the laboratory.
- Generate and configure a self-signed TLS certificate for HTTPS.
- Install SafeLine WAF and configure it as a reverse proxy.
- Expose DVWA through HTTPS on port 443 while forwarding requests to the DVWA backend on port 8080.
- Demonstrate SQL injection detection and blocking.
- Demonstrate advanced controls such as HTTP flood defense and custom IP denial.
- Collect screenshots and security-event evidence for analysis.

4. Scope

The project is limited to an isolated laboratory containing my own virtual machines and deliberately vulnerable DVWA instance. Testing is intended for controlled educational use. No production systems or third-party services are targeted.

5. Lab Environment

Component	Role	Configuration / Address
Kali Linux	Security testing client	192.168.79.128
Ubuntu Server	SafeLine host + Apache/DVWA + MySQL	192.168.79.132
SafeLine WAF	HTTPS reverse proxy / WAF	dvwa.local:443
Apache + DVWA	Vulnerable web application backend	192.168.79.132:8080

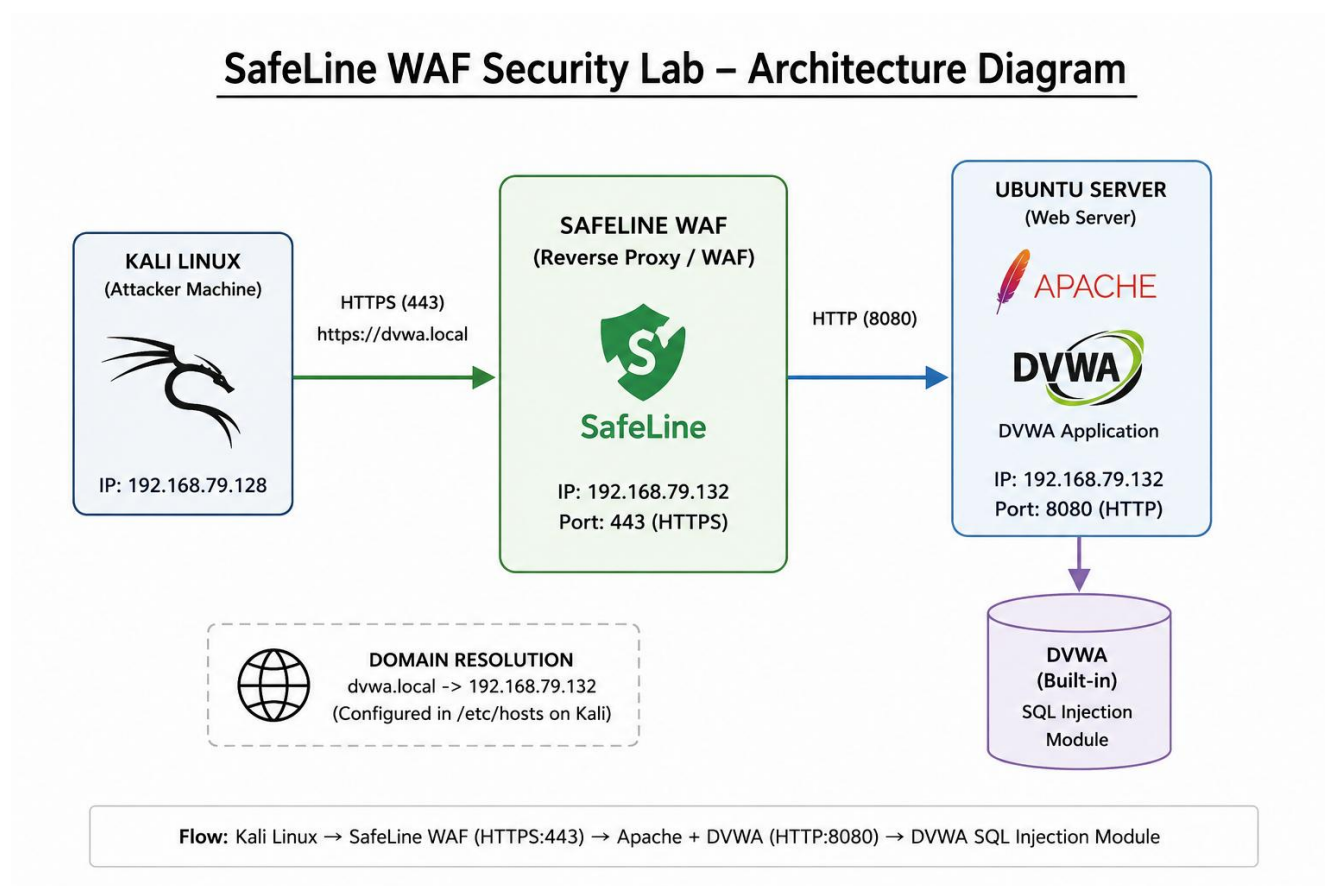


Figure 1: Lab architecture

6. Software and Technologies

- Ubuntu Server - server operating system.

- Apache HTTP Server 2.4.66 - web server.
- PHP 8.5.4 - application runtime.
- MySQL 8.4.10 - database server.
- DVWA - intentionally vulnerable web application.
- Kali Linux - security testing environment.
- SafeLine WAF - Web Application Firewall and reverse proxy.
- Docker - container runtime used by SafeLine.
- OpenSSL - self-signed TLS certificate generation.
- Firefox / browser - application access and verification.

7. Methodology

The laboratory followed a layered deployment approach. First, the application and database were configured and verified directly. Next, Apache was moved to port 8080. SafeLine was then installed and configured to listen on HTTPS port 443 and reverse proxy requests to the Apache backend. The local hostname dvwa.local was used for application access. After successful HTTPS login through SafeLine, controlled security tests were performed from Kali Linux.

8. Implementation and Configuration

8.1 Apache and DVWA Deployment

Apache was configured on the Ubuntu Server and DVWA was deployed under /var/www/html/dvwa. PHP and the required database connectivity were available for the DVWA application. The DVWA application was verified to be accessible through the Apache web server before configuring SafeLine WAF.

```
hooks@ubuntu: ~  
hooks@ubuntu:~$ sudo systemctl status apache2 --no-pager  
● apache2.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)  
   Active: active (running) since Thu 2026-08-27 09:05:47 UTC; 40s ago  
     Invocation: 42cff9c73d3f4083b4f45983daf3cc8a  
       Docs: https://httpd.apache.org/docs/2.4/  
    Main PID: 13977 (apache2)  
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"  
      Tasks: 6 (limit: 1034)  
     Memory: 11.6M (peak: 11.9M)  
        CPU: 131ms  
     CGroup: /system.slice/apache2.service  
             └─13977 /usr/sbin/apache2 -k start -DFOREGROUND  
               └─13980 /usr/sbin/apache2 -k start -DFOREGROUND  
                 └─13981 /usr/sbin/apache2 -k start -DFOREGROUND  
                   └─13982 /usr/sbin/apache2 -k start -DFOREGROUND  
                     └─13984 /usr/sbin/apache2 -k start -DFOREGROUND  
                       └─13985 /usr/sbin/apache2 -k start -DFOREGROUND  
  
Aug 27 09:05:47 ubuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...  
Aug 27 09:05:47 ubuntu apachectl[13977]: AH00558: apache2: Could not reliably determine the...ssage  
Aug 27 09:05:47 ubuntu systemd[1]: Started apache2.service - The Apache HTTP Server.  
Hint: Some lines were ellipsized, use -l to show in full.  
hooks@ubuntu:~$
```

Figure 2: Apache on Ubuntu server

8.2 DVWA Application Configuration

The DVWA application was configured and verified on the Ubuntu Server before integrating it with SafeLine WAF. The application was initially tested directly through the Apache backend and was later placed behind SafeLine as a reverse-proxy protected application.

The DVWA environment was used as the deliberately vulnerable target for the security testing performed in this project, including SQL injection and other controlled security tests.

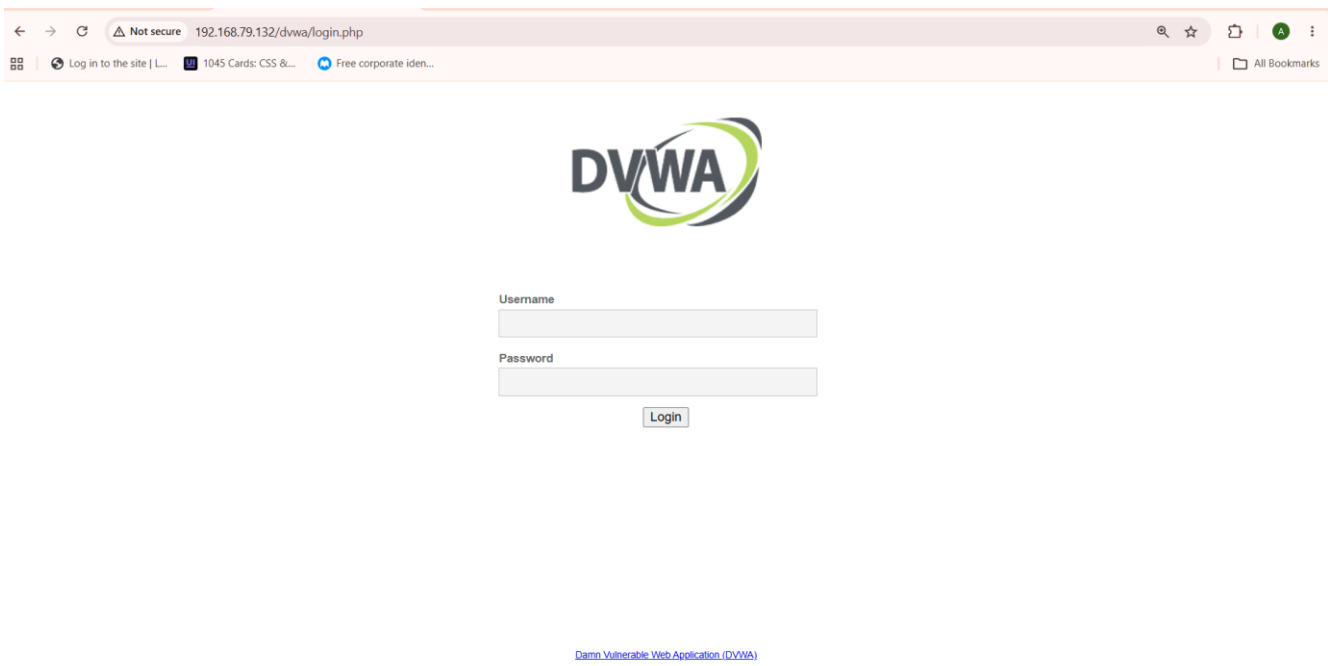


Figure 3: DVWA access before WAF

8.3 Moving Apache to Port 8080

Apache was changed from its default HTTP listening port to port 8080. The ports.conf and default virtual-host configuration were updated, followed by an Apache restart. This separated the application backend from the WAF-facing HTTPS listener.

Backend URL used for testing: <http://192.168.79.132:8080/dvwa/>

```
hooks@ubuntu: ~  
hooks@ubuntu:~$ sudo ss -tulpn | grep apache2  
tcp LISTEN 0      511                *:8080             *:~                users:(("apache2",pid=2734,fd=4),("apache2",pid=2733,fd=4),("apache2",pid=2732,fd=4),("apache2",pid=2731,fd=4),("apache2",pid=2730,fd=4),("apache2",pid=2729,fd=4),("apache2",pid=2632,fd=4),("apache2",pid=2631,fd=4),("apache2",pid=2627,fd=4),("apache2",pid=2625,fd=4),("apache2",pid=2622,fd=4))  
hooks@ubuntu:~$
```

Figure 4: Apache on 8080 port

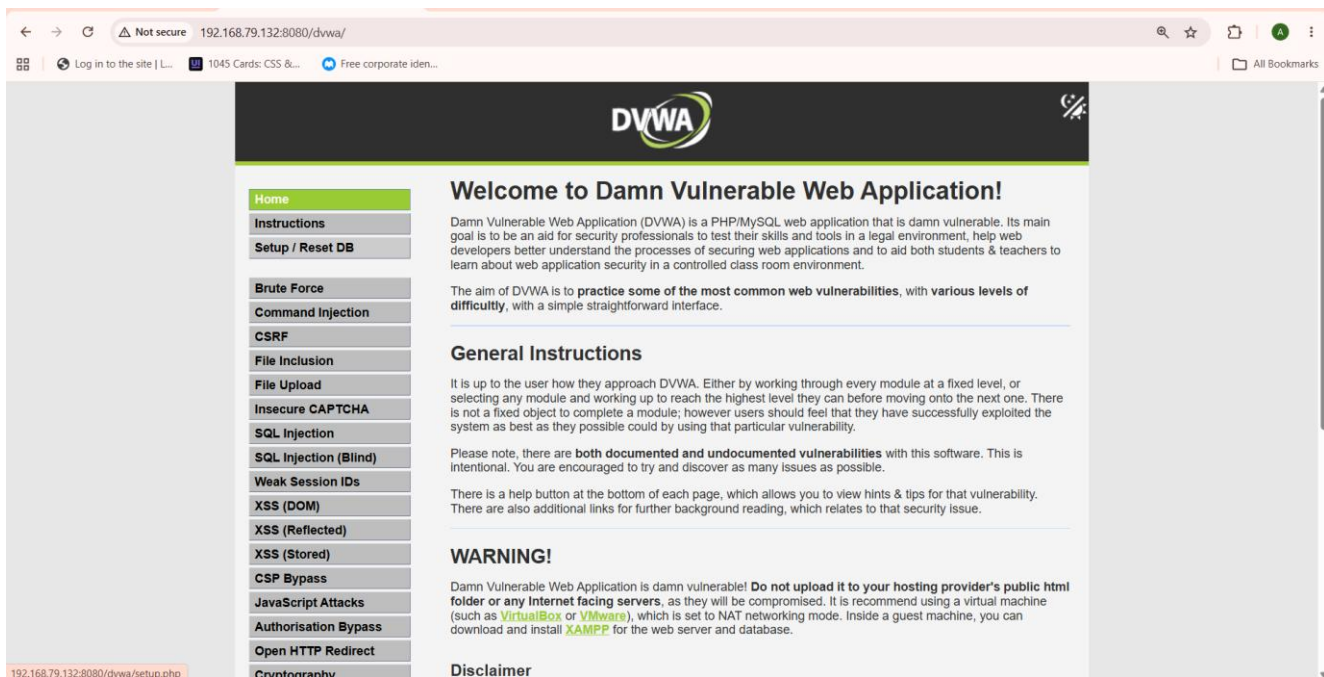


Figure 5: DVWA on 8080 port

8.4 Local DNS/Hostname Resolution

A local hosts-file mapping was used instead of deploying a dedicated BIND DNS server. The mapping used was:

```
192.168.79.132 dvwa.local
```

This allowed Kali Linux and the host browser to access the WAF using the hostname dvwa.local. The supplied guide describes /etc/hosts as the simple local-resolution method and presents BIND as optional.

```
root@hooks: ~
Session Actions Edit View Help
zsh: corrupt history file /root/.zsh_history
(root@hooks)-[~]
# nano /etc/hosts

(root@hooks)-[~]
# ping -c 3 dvwa.local
PING dvwa.local (192.168.79.132) 56(84) bytes of data:
64 bytes from dvwa.local (192.168.79.132): icmp_seq=1 ttl=64 time=7.39 ms
64 bytes from dvwa.local (192.168.79.132): icmp_seq=2 ttl=64 time=0.707 ms
64 bytes from dvwa.local (192.168.79.132): icmp_seq=3 ttl=64 time=223 ms

— dvwa.local ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2031ms
rtt min/avg/max/mdev = 0.707/77.177/223.433/103.454 ms

(root@hooks)-[~]
#
```

Figure 6: DVWA dns resolution from kali

8.5 Self-Signed HTTPS Certificate

A self-signed RSA 2048-bit certificate was generated with OpenSSL for the laboratory hostname dvwa.local. The certificate and private key were imported into SafeLine so the WAF could terminate HTTPS connections on port 443.

Certificate files used during configuration: /etc/ssl/dvwa/dvwa.crt and /etc/ssl/dvwa/dvwa.key

```
hooks@ubuntu: ~
hooks@ubuntu:~$ sudo ls -l /etc/ssl/dvwa/
total 8
-rw-r--r-- 1 root root 1302 Aug 30 09:38 dvwa.crt
-rw----- 1 root root 1704 Aug 30 09:37 dvwa.key
hooks@ubuntu:~$
```

Figure 7: DVWA ssl certificate

8.6 SafeLine WAF Installation

SafeLine was installed on the Ubuntu Server using its automated deployment method. The installation created the SafeLine containerized services and provided access to the SafeLine management interface.

```
hooks@ubuntu: ~  
✓Image chaitin/safeline-postgres:15.18 Pulled 92.3s  
✓Image chaitin/safeline-tengine-g:latest Pulled 90.0s  
✓Image chaitin/safeline-luigi-g:latest Pulled 58.9s  
✓Image chaitin/safeline-fvm-g:latest Pulled 77.5s  
✓Image chaitin/safeline-chaos-g:latest Pulled 90.4s  
✓Image chaitin/safeline-mgt-g:latest Pulled 58.6s  
✓Image chaitin/safeline-detector-g:latest Pulled 76.9s  
✓Container safeline-tengine Started 2.8s  
✓Network safeline-ce Created 1.8s  
✓Container safeline-detector Started 1.9s  
✓Container safeline-pg Started 1.8s  
✓Container safeline-fvm Started 1.8s  
✓Container safeline-chaos Started 1.9s  
✓Container safeline-mgt Started 3.5s  
✓Container safeline-luigi Started 8.6s  
https://discord.gg/BDUa5UQnTW  
  
Visit the above address to join the SafeLine discord group  
[SafeLine] Waiting for safeline-pg to be healthy  
[SafeLine] Container safeline-pg is healthy  
[SafeLine] Waiting for safeline-mgt to be healthy  
[SafeLine] Container safeline-mgt is healthy  
[INFO] Initial username:  
[INFO] Initial password:  
[INFO] Done  
[SafeLine] SafeLine WAF has been installed, visit the following address to Web admin console  
[SafeLine] https://0.0.0.0:9443/  
[SafeLine] https://192.168.79.132:9443/  
[SafeLine] https://127.0.0.1:9443/  
hooks@ubuntu:~$
```

Figure 8: Safeline installation

```
hooks@ubuntu: ~  
hooks@ubuntu:~$ sudo docker ps  
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS  
330d1c91e92e   chaitin/safeline-luigi-g:latest     "/bin/sh -c /app/lui..." 6 minutes ago  Up 6 minutes  80/tcp  
ale22a3962bc   safeline-luigi                      "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  80/tcp, 0.0.0.0:9443->1443/tcp, [::]:9443->1443/tcp  
443->1443/tcp   chaitin/safeline-mgt-g:latest       "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  8080/tcp, 9000/tcp  
8fe4bddd92     safeline-mgt                        "/entrypoint.sh /us..." 6 minutes ago  Up 6 minutes  8080/tcp, 9000/tcp  
05c043897b35   chaitin/safeline-chaos-g:latest     "/entrypoint.sh /us..." 6 minutes ago  Up 6 minutes  8080/tcp, 9000/tcp  
safeline-chaos                                "/entrypoint.sh /us..." 6 minutes ago  Up 6 minutes  8080/tcp, 9000/tcp  
eae50cc43ce9   chaitin/safeline-postgres:15.18     "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  5432/tcp  
safeline-pg                                    "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  5432/tcp  
9ab1314e645c   chaitin/safeline-fvm-g:latest       "/app/fvm /app/confi..." 6 minutes ago  Up 6 minutes  5432/tcp  
safeline-fvm                                    "app/fvm /app/confi..." 6 minutes ago  Up 6 minutes  5432/tcp  
9a93ed1ffdb8   chaitin/safeline-detector-g:latest  "/detector/entrypoin..." 6 minutes ago  Up 6 minutes  8000-8001/tcp  
safeline-detector                             "detector/entrypoin..." 6 minutes ago  Up 6 minutes  8000-8001/tcp  
9a93ed1ffdb8   chaitin/safeline-tengine-g:latest   "entrypoint.sh nginx..." 6 minutes ago  Up 6 minutes  8000-8001/tcp  
safeline-tengine                             "entrypoint.sh nginx..." 6 minutes ago  Up 6 minutes  8000-8001/tcp  
hooks@ubuntu:~$
```

Figure 9: Safeline Docker container

8.7 SafeLine Reverse Proxy Configuration

The DVWA application was added to SafeLine as a reverse-proxy application with the following configuration:

Setting	Value
Application / Domain	dvwa.local
Public listener	443 / HTTPS
Access method	Reverse Proxy
Upstream	http://192.168.79.132:8080
TLS certificate	Self-signed certificate for dvwa.local

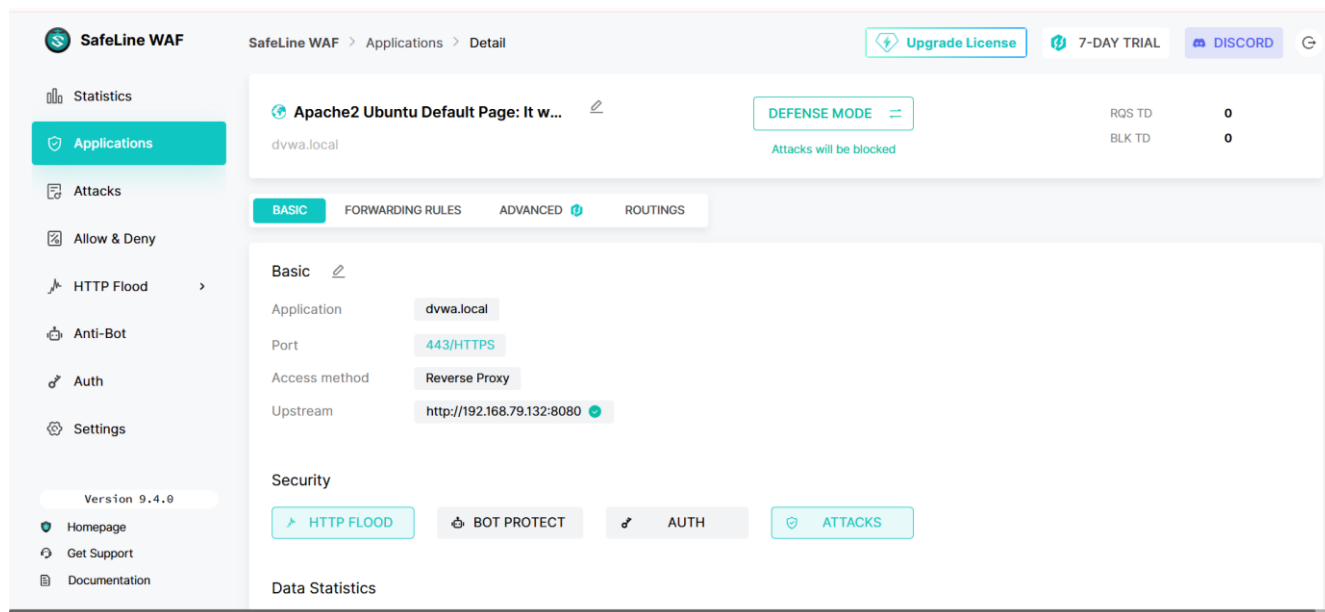


Figure 10: Safeline DVWA application config

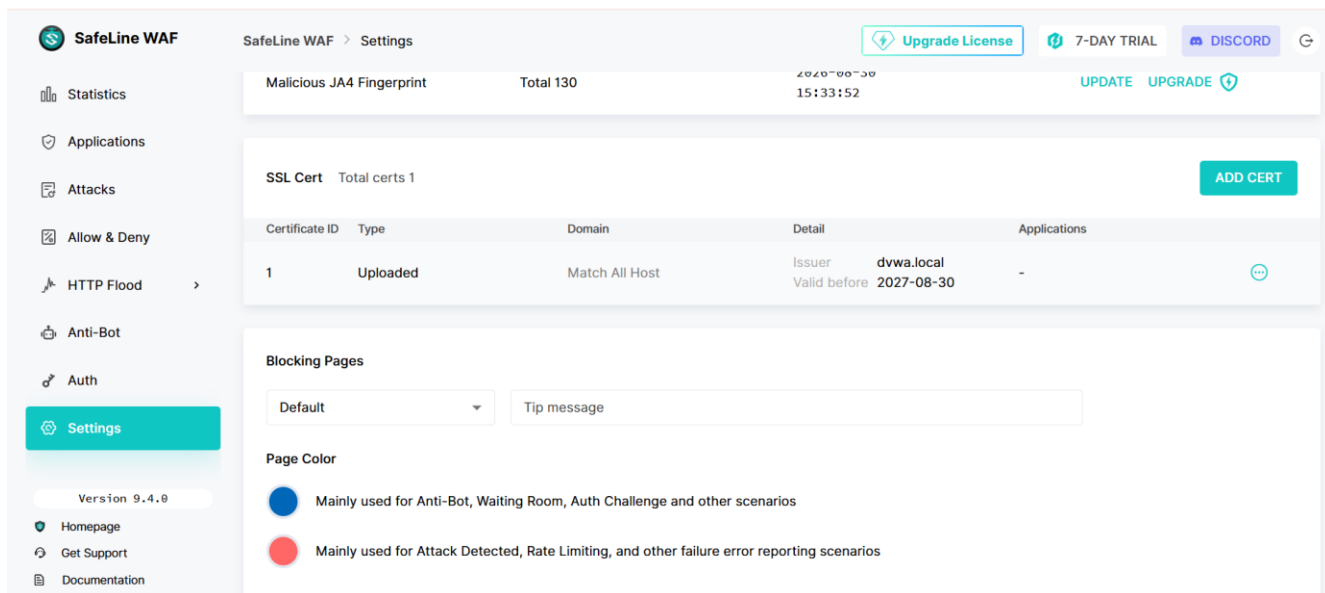


Figure 11: SSL certificates imported on DVWA

8.8 Final HTTPS Access

After configuration, DVWA was successfully accessed through the protected URL:

`https://dvwa.local/dvwa/`

The browser connection reached SafeLine on port 443, while SafeLine forwarded the request to the Apache/DVWA backend on port 8080.

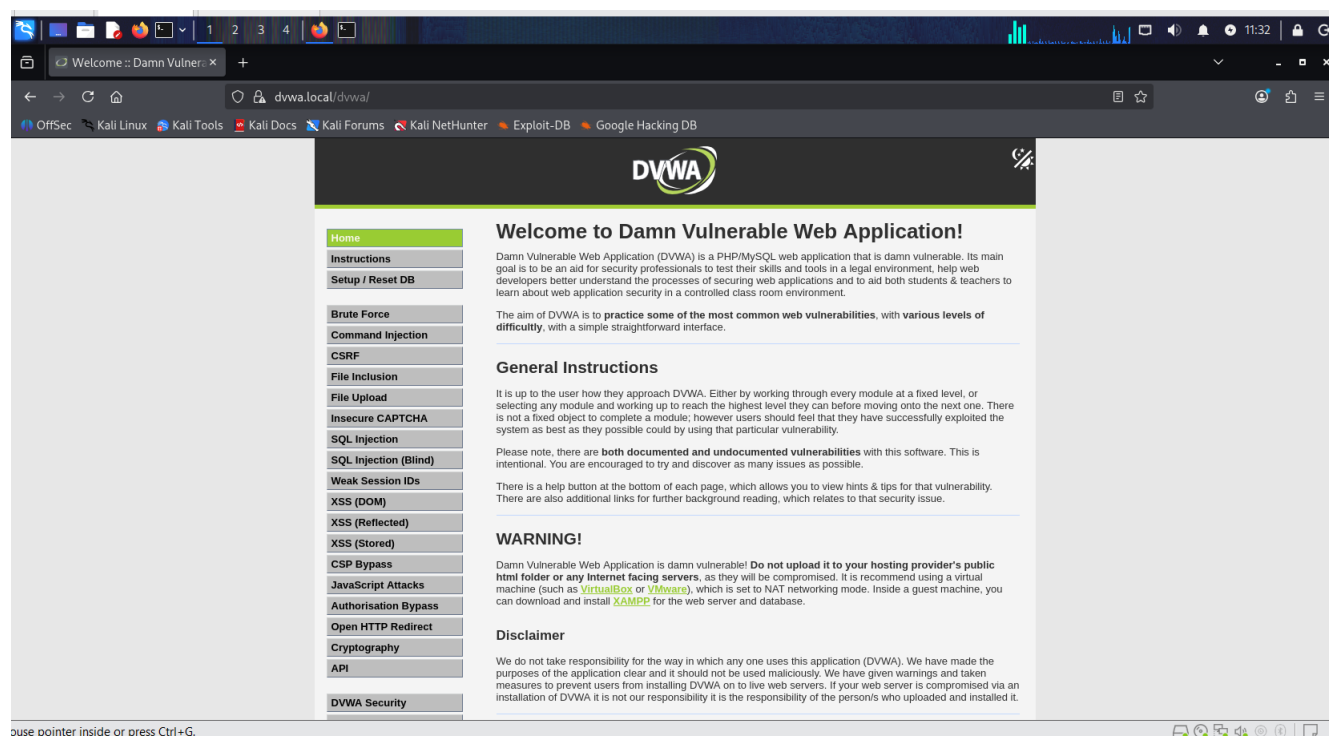


Figure 12: DVWA safeline protected

9. Security Testing

9.1 Baseline Application Test

Before security testing, DVWA was verified to be accessible through the SafeLine HTTPS endpoint. The DVWA login was successfully completed after resolving reverse-proxy session behavior. Direct backend access on port 8080 was also verified during troubleshooting.

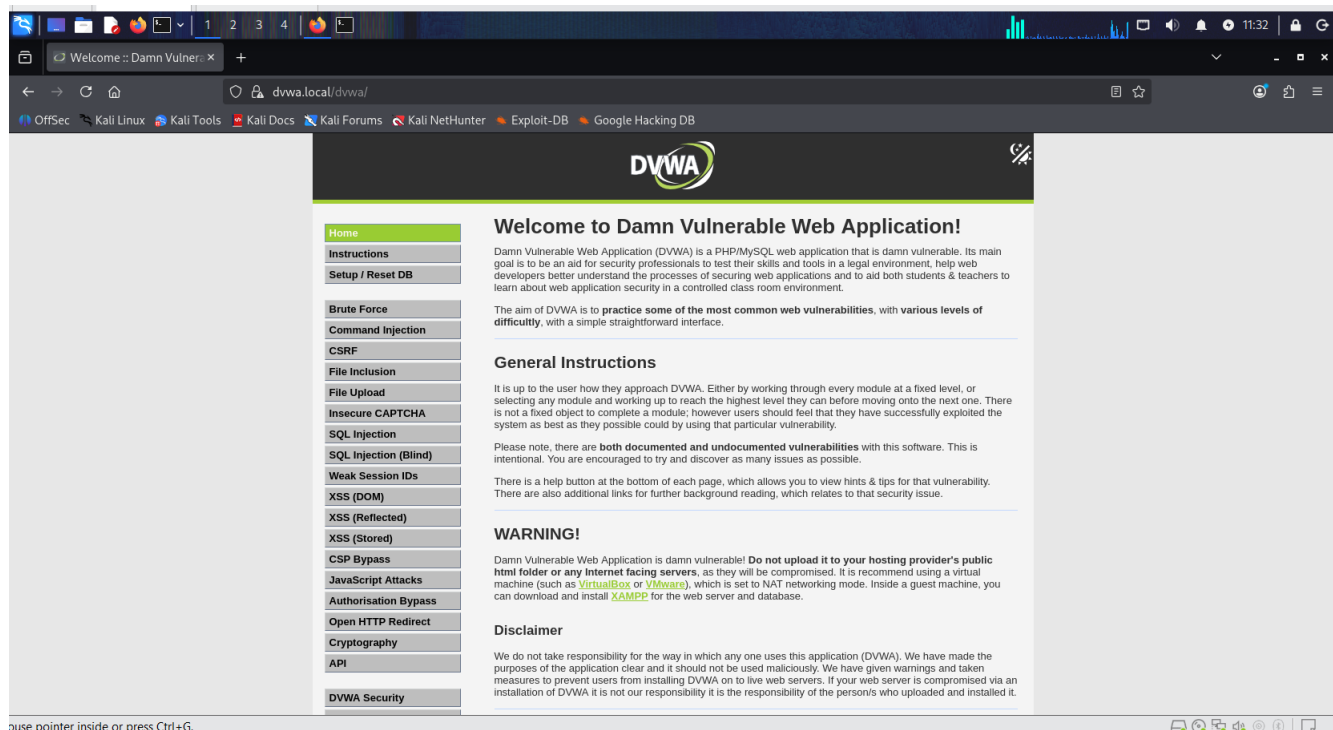


Figure 13: DVWA through safeline

9.2 SQL Injection Test

The DVWA security level was set to Low for the controlled vulnerability demonstration. A typical SQL injection expression was submitted to the DVWA SQL Injection functionality. The supplied guide uses the example `admin' OR '1'='1` for this demonstration.

In the completed test, SafeLine detected the malicious SQL injection request and blocked it. This demonstrated the WAF's ability to inspect application-layer requests and prevent the attack from reaching the vulnerable application.

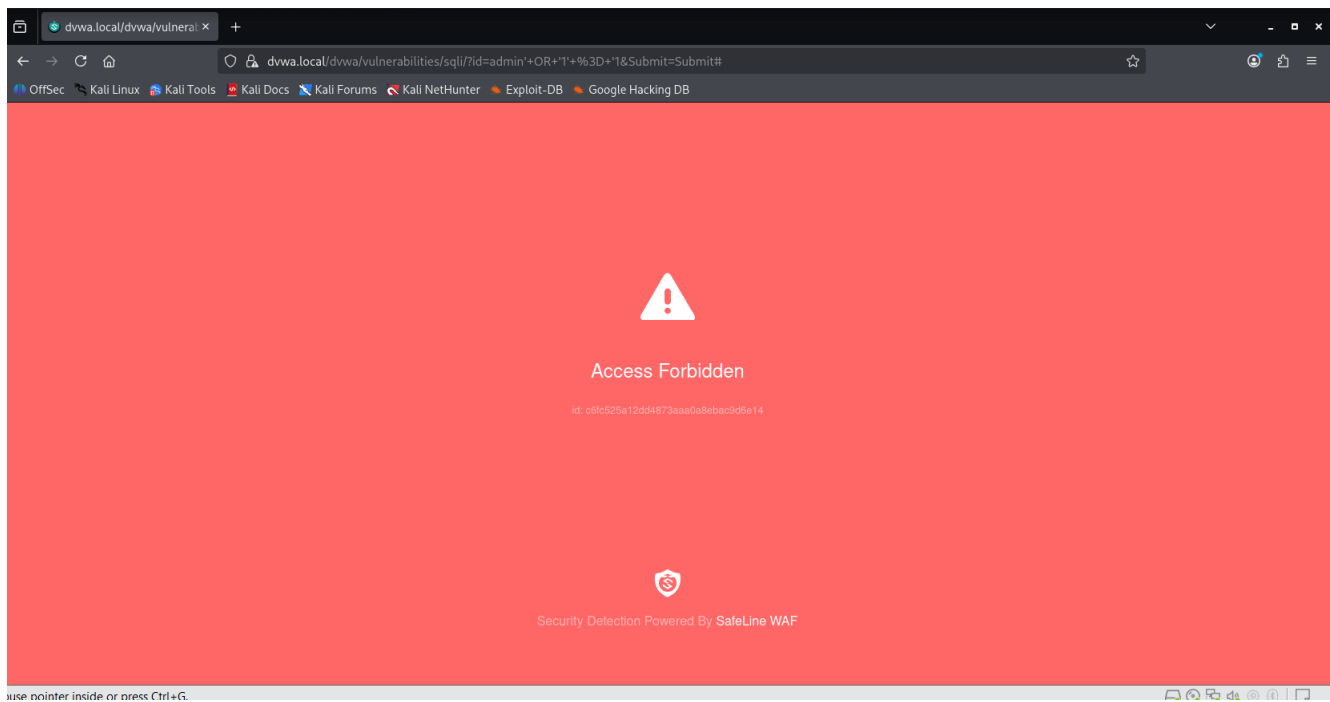


Figure 14: SQL injection attempt

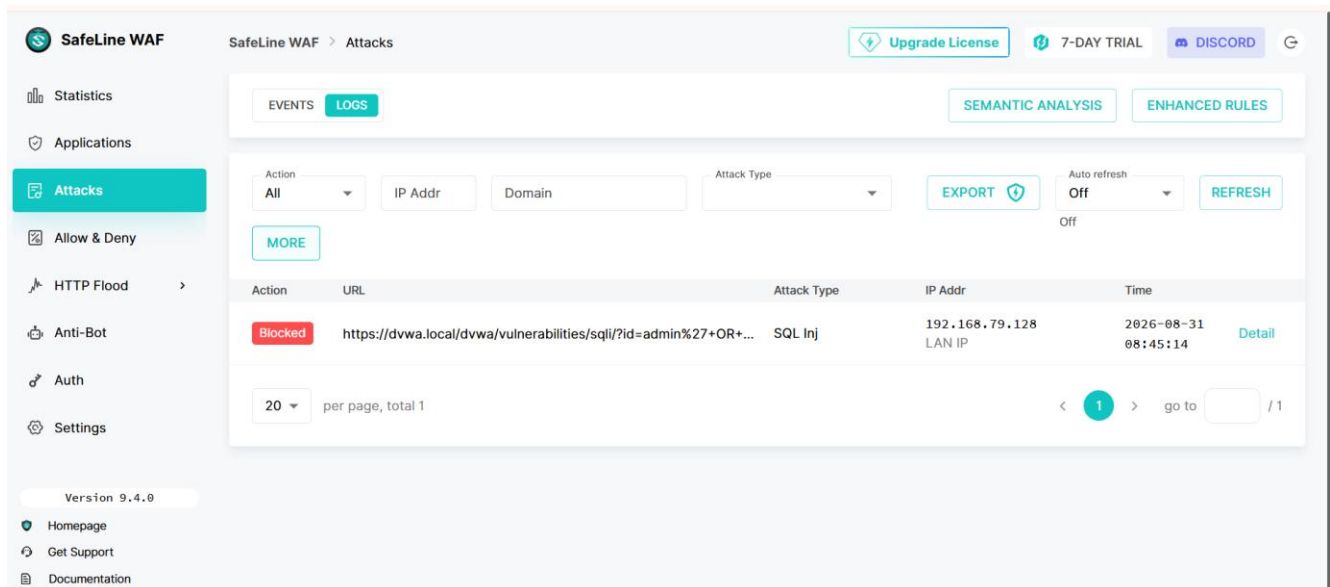


Figure 15: Safeline SQL alert

Result: PASS - SQL injection request was blocked by SafeLine.

9.3 HTTP Flood Defense

SafeLine HTTP Flood/DoS protection was configured and tested using controlled request generation from the Kali Linux lab machine. The test was designed to remain within the capacity of the small virtualized environment.

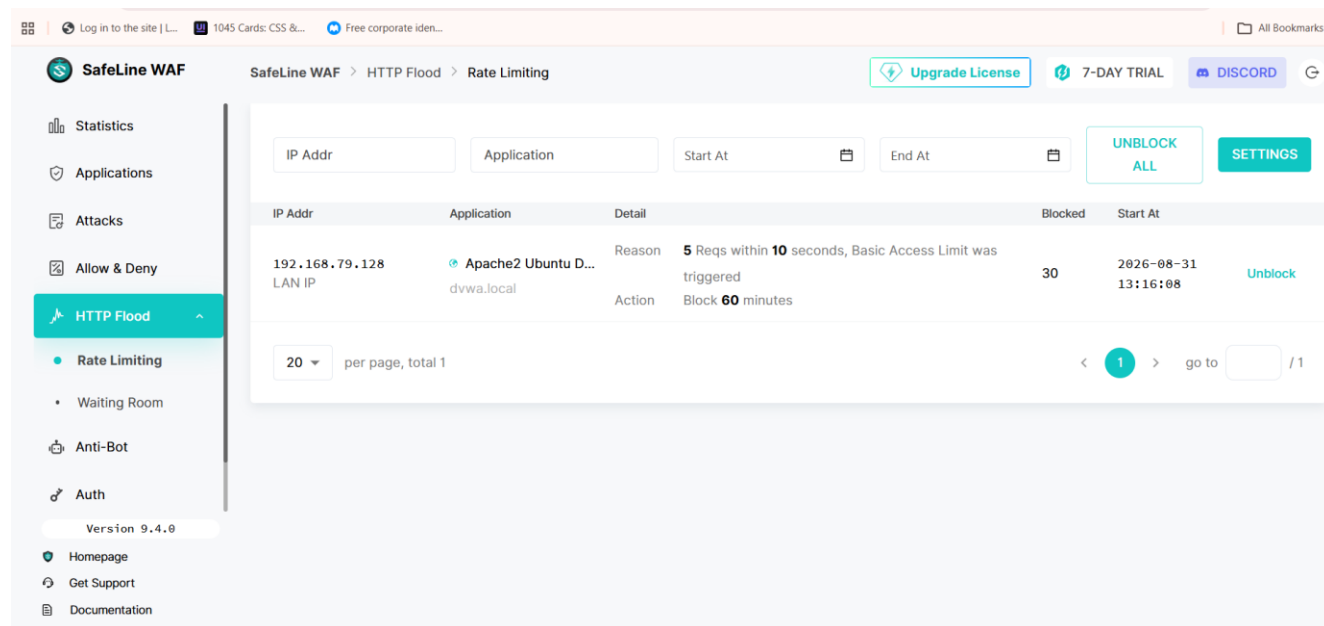


Figure 16: Safeline flood detection

Result: PASS - controlled excessive-request traffic was tested against the configured WAF protection.

9.4 Custom IP Deny Rule

A custom deny rule was configured for the Kali testing IP. This demonstrated that SafeLine can apply explicit source-IP access control in addition to its application-layer detection mechanisms. After verification, the temporary rule was disabled/removed so normal lab access could be restored.

Add rules
×

☐ Allow

☒ Deny Rule

Name *

attacker

Insert Position

First

Match Target

Source IP

Operator *

Equals

Content *

192.168.79.128 x

e.g: 192.168.10.10

ADD AN AND CONDITION

REMOVE THIS CONDITION GROUP

ADD AN OR CONDITION

☒ Enabled

CANCEL

SUBMIT

Figure 17: Custom IP deny rule

SafeLine WAF

SafeLine WAF > Allow & Deny

Upgrade License

7-DAY TRIAL

DISCORD

G

EVENTS
LOGS

CUSTOM RULES

Action

All

IP Addr

Domain

Type

Auto refresh

Off

REFRESH

MORE

Action	Address	Type	Rule Name	Attack IP	Time	
Blocked	https://dvwa.local/dvwa/	Deny Rule	attacker	192.168.79.128 LAN IP	2026-08-31 13:22:56	Detail
Blocked	https://dvwa.local/dvwa/	Deny Rule	attacker	192.168.79.128 LAN IP	2026-08-31 13:21:17	Detail
Blocked	https://dvwa.local/dvwa/	Deny Rule	attacker	192.168.79.128 LAN IP	2026-08-31 13:20:52	Detail

20

per page, total 3

<

1

>

go to
/ 1

Figure 18 : Reuest blocked by IP deny rule

Result: PASS – the configured source IP was denied as expected.

10. Results and Analysis

Test	Expected Behavior	Observed Result	Status
Normal HTTPS access	DVWA loads through SafeLine	DVWA accessible at https://dvwa.local/dvwa/	PASS
DVWA login	Session works through reverse proxy	Login successfully completed	PASS
SQL Injection	WAF detects/blocks malicious request	SafeLine blocked the request	PASS
HTTP Flood Defense	Excessive request traffic detected/limited	Controlled test detected/blocked	PASS
Custom IP Deny	Configured source IP denied	Kali source was blocked	PASS

The results demonstrate the difference between direct application access and protected access. The Apache/DVWA backend remained available on port 8080, while client traffic intended for the application was directed through SafeLine over HTTPS. The successful SQL injection block provides the clearest demonstration of application-layer protection. The advanced tests further demonstrate rate-based and source-based controls.

11. Security Architecture and Defense Flow

The defensive flow is designed so that the vulnerable application is not the first point exposed to the testing client. SafeLine receives the HTTP(S) request, applies its configured security controls, and forwards permitted traffic to the backend.

For a malicious request, the intended flow is:

- Kali Linux sends a request to <https://dvwa.local/dvwa/>.
- SafeLine receives the request on HTTPS port 443.
- SafeLine analyzes the HTTP request.
- If the request matches a protected attack pattern or configured deny condition, SafeLine blocks it and records a security event.
- If permitted, SafeLine forwards the request to Apache/DVWA on port 8080.
- DVWA processes legitimate traffic normally.

12. Limitations

- The laboratory uses a self-signed certificate, so browsers do not inherently trust the certificate.
- The environment is virtualized and has limited memory compared with a production deployment.
- The tests are controlled demonstrations and do not represent full production penetration testing.
- Local `/etc/hosts` resolution was used instead of a production DNS service.
- DVWA is intentionally vulnerable and must remain isolated from production or public networks.

13. Conclusion

The SafeLine WAF Security Lab successfully demonstrated the deployment of a vulnerable web application behind a Web Application Firewall. DVWA was hosted on Ubuntu with Apache and MySQL, while SafeLine provided the HTTPS reverse-proxy layer. Moving Apache to port 8080 allowed SafeLine to act as the front-end security control on port 443. The use of dvwa.local and a self-signed certificate provided a realistic HTTPS testing workflow within the isolated lab.

The security tests demonstrated that SafeLine could detect and block the selected SQL injection request. Additional testing of HTTP flood protection and custom source-IP denial demonstrated further defensive controls. Overall, the project provides a practical illustration of how WAF controls can reduce the exposure of a vulnerable application and how security events can be observed during testing.

14. Future Work

- Test additional DVWA vulnerability classes such as XSS, file inclusion, and command injection in the isolated lab.
- Integrate SafeLine events with a SIEM such as Wazuh for centralized monitoring and correlation.
- Compare detection behavior at different DVWA security levels.
- Add additional vulnerable applications such as OWASP Juice Shop.
- Develop repeatable traffic-generation and evidence-collection scripts.
- Use a trusted internal CA for a cleaner HTTPS experience in a larger laboratory.

15. References

1. Supplied laboratory guide: Safeline WAF Guide(1).pdf - used as the primary project procedure and scope reference.
2. DVWA project documentation - for DVWA deployment and application behavior.
3. SafeLine documentation - for WAF deployment, reverse proxy, certificate, and protection configuration.